Silent Payments

BIP 352

Agenda

- The static-address problem

Agenda

- The static-address problem
- Prior art: Stealth addresses, BIP47, BIP351

Agenda

- The static-address problem
- Prior art: Stealth addresses, BIP47, BIP351
- BIP 352's history and status

Agenda

- The static-address problem
- Prior art: Stealth addresses, BIP47, BIP351
- BIP 352's history and status
- Sender derivation and receiver scanning

Agenda

- The static-address problem
- Prior art: Stealth addresses, BIP47, BIP351
- BIP 352's history and status
- Sender derivation and receiver scanning
- Trade-offs and constraints regarding wallets

Agenda

- The static-address problem
- Prior art: Stealth addresses, BIP47, BIP351
- BIP 352's history and status
- Sender derivation and receiver scanning
- Trade-offs and constraints regarding wallets

Privacy problems

A donation address is a graph node

Every payment to the address is trivially linked

Fresh addresses require a delivery channel

- Best practice: give every sender a fresh address

Fresh addresses require a delivery channel

- Best practice: give every sender a fresh address
- But that generally needs an interaction

Fresh addresses require a delivery channel

- Best practice: give every sender a fresh address
- But that generally needs an interaction
- A public static address removes the interaction but creates permanent on-chain linkage

Fresh addresses require a delivery channel

- Best practice: give every sender a fresh address
- But that generally needs an interaction
- A public static address removes the interaction but creates permanent on-chain linkage
- Silent Payments aim for both

Fresh addresses require a delivery channel

- Best practice: give every sender a fresh address
- But that generally needs an interaction
- A public static address removes the interaction but creates permanent on-chain linkage
- Silent Payments aim for both
  - one reusable public identifier
  - one unique on-chain output per payment

Goals

Reusable
publish one identifier

Unlinked
each payment has a different output key

Non-interactive
no invoice or notification round trip

The hard part is letting the recipient find the payment without giving observers the same signal.

Prior art

Le galerie

Static address
no setup, payments link

Stealth address
unique output, ephemeral-key signal

BIP47
unique outputs, notification step

BIP352
unique outputs, scan every candidate

Classic stealth addresses

- Long-standing construction: recipient publishes a public key or key pair

Classic stealth addresses

- Long-standing construction: recipient publishes a public key or key pair
- Sender creates an ephemeral key and derives a one-time destination with ECDH

Classic stealth addresses

- Long-standing construction: recipient publishes a public key or key pair
- Sender creates an ephemeral key and derives a one-time destination with ECDH
- Recipient needs the sender's ephemeral public key to discover the payment

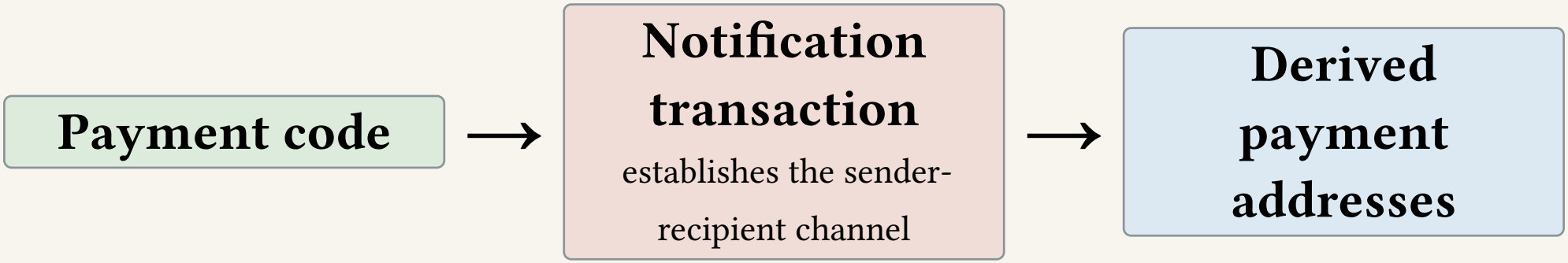
Classic stealth addresses

- Long-standing construction: recipient publishes a public key or key pair
- Sender creates an ephemeral key and derives a one-time destination with ECDH
- Recipient needs the sender's ephemeral public key to discover the payment
- That key must be carried on-chain or otherwise delivered

Classic stealth addresses

- Long-standing construction: recipient publishes a public key or key pair
- Sender creates an ephemeral key and derives a one-time destination with ECDH
- Recipient needs the sender's ephemeral public key to discover the payment
- That key must be carried on-chain or otherwise delivered
- Silent Payments retain one-time ECDH derivation (Elliptic Curve Diffie-Helman)
  - but reuse the transaction inputs as the public data instead of adding an announcement

BIP47 reusable payment codes



BIP47 avoids address reuse, but the one-time notification adds cost and a visible protocol footprint.

BIP47 and BIP352

Property	BIP47	BIP352
Public recipient identifier	Payment code	Silent payment address
First payment	On-chain notification	Direct payment
On-chain recognizability	Notification is recognizable	No special marker
Recipient work	Watch derived addresses	Scan eligible transactions
Output type	Protocol-dependent	Taproot v1
Sender linkage	Recipient learns sender channel	No persistent channel

BIP351

- 2022 informational proposal by Alfred Hodler and Clark Moody

BIP351

- 2022 informational proposal by Alfred Hodler and Clark Moody
- Also uses reusable payment codes and derived payment addresses

BIP351

- 2022 informational proposal by Alfred Hodler and Clark Moody
- Also uses reusable payment codes and derived payment addresses
- Sender publishes a 40-byte `OP_RETURN` notification to establish a channel

BIP351

- 2022 informational proposal by Alfred Hodler and Clark Moody
- Also uses reusable payment codes and derived payment addresses
- Sender publishes a 40-byte `OP_RETURN` notification to establish a channel
- Adds explicit address-type flags to the payment code

BIP351

- 2022 informational proposal by Alfred Hodler and Clark Moody
- Also uses reusable payment codes and derived payment addresses
- Sender publishes a 40-byte `OP_RETURN` notification to establish a channel
- Adds explicit address-type flags to the payment code
- Like BIP47, it optimizes recipient scanning with a notification step

BIP351

- 2022 informational proposal by Alfred Hodler and Clark Moody
- Also uses reusable payment codes and derived payment addresses
- Sender publishes a 40-byte `OP_RETURN` notification to establish a channel
- Adds explicit address-type flags to the payment code
- Like BIP47, it optimizes recipient scanning with a notification step
- BIP352 instead avoids that step and pays for recipient-side transaction scanning

BIP 352

History

1. **2015** - Justus Ranvier proposes BIP47 reusable payment codes
2.
3.
4.
5.

History

1. **2015** - Justus Ranvier proposes BIP47 reusable payment codes
2. **13 March 2022** - Ruben Somsen publishes the original Silent Payments proposal
3.
4.
5.

History

1. **2015** - Justus Ranvier proposes BIP47 reusable payment codes
2. **13 March 2022** - Ruben Somsen publishes the original Silent Payments proposal
3. **28 March 2022** - public bitcoin-dev discussion begins
4.
5.

History

1. **2015** - Justus Ranvier proposes BIP47 reusable payment codes
2. **13 March 2022** - Ruben Somsen publishes the original Silent Payments proposal
3. **28 March 2022** - public bitcoin-dev discussion begins
4. **9 March 2023** - assigned BIP 352
5.

History

1. **2015** - Justus Ranvier proposes BIP47 reusable payment codes
2. **13 March 2022** - Ruben Somsen publishes the original Silent Payments proposal
3. **28 March 2022** - public bitcoin-dev discussion begins
4. **9 March 2023** - assigned BIP 352
5. **8 May 2024** - BIP 352 v1.0 merged

History

1. **2015** - Justus Ranvier proposes BIP47 reusable payment codes
2. **13 March 2022** - Ruben Somsen publishes the original Silent Payments proposal
3. **28 March 2022** - public bitcoin-dev discussion begins
4. **9 March 2023** - assigned BIP 352
5. **8 May 2024** - BIP 352 v1.0 merged

BIP 352

- Application-layer payment protocol

BIP 352

- Application-layer payment protocol
- Specification status: **Complete**

BIP 352

- Application-layer payment protocol
- Specification status: **Complete**
- Authors: josibake, Ruben Somsen, Sebastian Falbesoner

BIP 352

- Application-layer payment protocol
- Specification status: **Complete**
- Authors: josibake, Ruben Somsen, Sebastian Falbesoner
- Uses BIP340/341 Taproot and Bech32m primitives

BIP 352

- Application-layer payment protocol
- Specification status: **Complete**
- Authors: josibake, Ruben Somsen, Sebastian Falbesoner
- Uses BIP340/341 Taproot and Bech32m primitives
- No consensus change and no special transaction output

BIP 352

- Application-layer payment protocol
- Specification status: **Complete**
- Authors: josibake, Ruben Somsen, Sebastian Falbesoner
- Uses BIP340/341 Taproot and Bech32m primitives
- No consensus change and no special transaction output
  - Therefore no soft fork lol

BIP 352

- Application-layer payment protocol
- Specification status: **Complete**
- Authors: josibake, Ruben Somsen, Sebastian Falbesoner
- Uses BIP340/341 Taproot and Bech32m primitives
- No consensus change and no special transaction output
  - Therefore no soft fork lol
- A silent payment output is an ordinary single-key Taproot output

Address format

Silent payment address

`sp1q...` = Bech32m(scan public key `B_scan` || spend public key `B_spend`)

- The scan key lets an online wallet detect payments

Address format

Silent payment address

`sp1q...` = Bech32m(scan public key `B_scan` || spend public key `B_spend`)

- The scan key lets an online wallet detect payments
- The spend key controls the received funds

Address format

Silent payment address

`sp1q...` = Bech32m(scan public key `B_scan` || spend public key `B_spend`)

- The scan key lets an online wallet detect payments
- The spend key controls the received funds
- Separating them allows scanning without keeping spending material online

Address format

Silent payment address

`sp1q...` = Bech32m(scan public key `B_scan` || spend public key `B_spend`)

- The scan key lets an online wallet detect payments
- The spend key controls the received funds
- Separating them allows scanning without keeping spending material online
- Version 0 sends only to Taproot outputs
- The addresses are pretty long btw, 116 chars (117 on testnet)

Two keys

Scan key

online secret used to test candidate transactions

Spend key

offline secret, tweaked to spend each received output

If the scan key is stolen, the attacker gets discovery, but not control of the funds

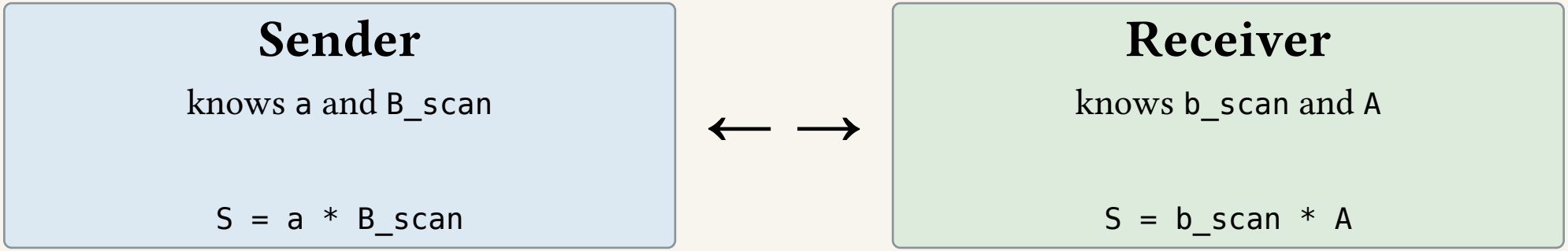
Explanation

Variables

Symbol	Meaning
`a`	sender private input key / scalar
`A = a * G`	sender public key / curve point
`b_scan`	recipient scan private key
`B_scan`	recipient scan public key
`b_spend`	recipient spend private key
`B_spend`	recipient spend public key
`S`	shared secret curve point
`t`	payment-specific tweak
`P`	one-time Taproot output key
`G`	secp256k1 generator point

- lowercase (a,b) = private scalar, uppercase (A,B) = public curve point

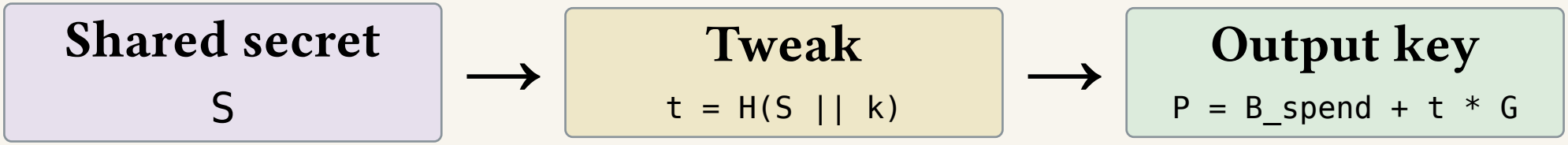
1. shared secret derivation



Since `A = a * G` and `B_scan = b_scan * G`, both sides derive the same `S`.

BIP352 also binds this derivation to the concrete transaction inputs.

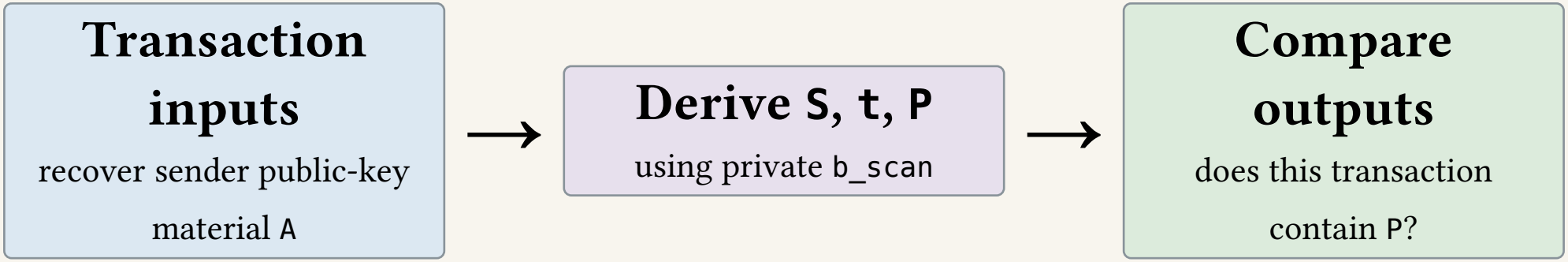
2. Shared secret computes a unique output



`k` starts at 0 and allows several outputs to the same recipient in one transaction.

The sender pays to `P` as an ordinary Taproot output.

3. receiver finds the payment



If `P` appears among the Taproot outputs, the receiver knows that output belongs to him.

Why silent?

Public information already in the transaction

sender input keys

→

Ordinary Taproot output `P`

no notifs anywhere onchain

The sender constructs `P`; the receiver independently rediscovers it.

Spending it is a regular Taproot key-path spend with recipient's spend key plus the same tweak.

Trade-offs

Trade-offs

- Receiver-side scanning is way heavier than conventional wallet scanning

Trade-offs

- Receiver-side scanning is way heavier than conventional wallet scanning
  - each eligible candidate transaction requires elliptic-curve calcs

Trade-offs

- Receiver-side scanning is way heavier than conventional wallet scanning
  - each eligible candidate transaction requires elliptic-curve calcs
- Full-node scanning is straightforward, but light-client support is probably more involved

Trade-offs

- Receiver-side scanning is way heavier than conventional wallet scanning
  - each eligible candidate transaction requires elliptic-curve calcs
- Full-node scanning is straightforward, but light-client support is probably more involved
- Both sender and recipient wallets must implement the protocol

Trade-offs

- Receiver-side scanning is way heavier than conventional wallet scanning
  - each eligible candidate transaction requires elliptic-curve calcs
- Full-node scanning is straightforward, but light-client support is probably more involved
- Both sender and recipient wallets must implement the protocol
- V0 requires Taproot outputs

Trade-offs

- Receiver-side scanning is way heavier than conventional wallet scanning
  - each eligible candidate transaction requires elliptic-curve calcs
- Full-node scanning is straightforward, but light-client support is probably more involved
- Both sender and recipient wallets must implement the protocol
- V0 requires Taproot outputs
- Collaborative transaction support is not currently recommended
  - no formal security proof for the required coordination

Scanning

Classic wallet

look up known scripts in UTXO set or filters

Silent payment wallet

derive a candidate key from each eligible transaction

Q&A